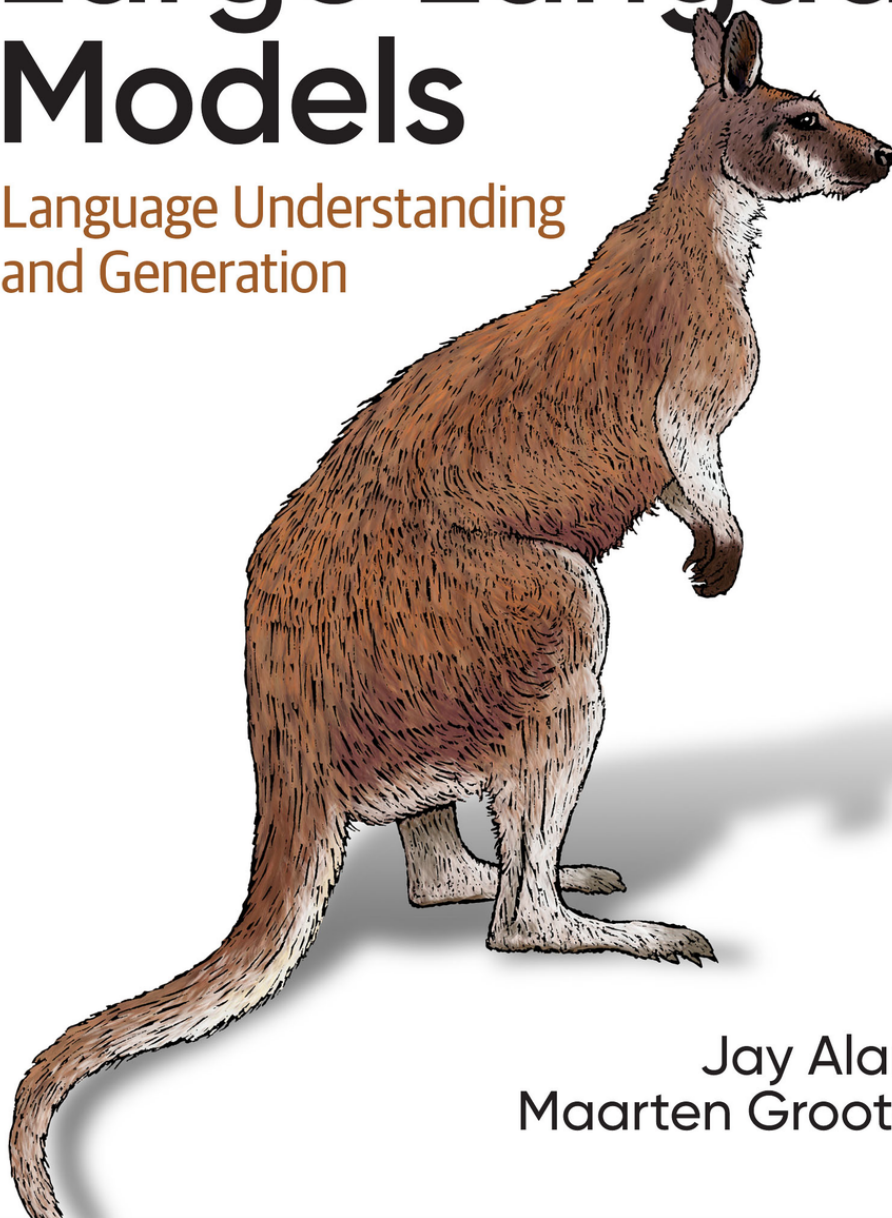


O'REILLY®

Hands-On Large Language Models

Language Understanding
and Generation



Jay Alammar &
Maarten Grootendorst

Praise for *Hands-On Large Language Models*

This is an exceptional guide to the world of language models and their practical applications in industry. Its highly-visual coverage of generative, representational, and retrieval applications of language models empowers readers to quickly understand, use, and refine LLMs. Highly recommended!

—Nils Reimers, Director of Machine Learning at Cohere | creator of sentence-transformers

Jay and Maarten have continued their tradition of providing beautifully illustrated and insightful descriptions of complex topics in their new book. Bolstered with working code, timelines, and references to key papers, their book is a valuable resource for anyone looking to understand the main techniques behind how Large Language Models are built.

—Andrew Ng, founder of DeepLearning.AI

I can't think of another book that is more important to read right now. On every single page, I learned something that is critical to success in this era of language models.

—Josh Starmer, StatQuest

If you're looking to get up to speed in everything regarding LLMs, look no further! In this wonderful book, Jay and Maarten will take you from zero to expert in the history and latest advances in large language models. With very intuitive explanations, great real-life examples, clear illustrations, and comprehensive code labs, this book lifts the curtain on the complexities of transformer models, tokenizers, semantic search, RAG, and many other cutting-edge technologies. A must read for anyone interested in the latest AI technology!

—Luis Serrano, PhD, Founder and CEO of Serrano Academy

This book is a must-read for anyone interested in the rapidly-evolving field of generative AI. With a focus on both text and visual embeddings, it's a great blend of algorithmic evolution, theoretical rigor, and practical guidance. Whether you are a student, researcher, or industry professional, this book will equip you with the use cases and solutions needed to level-up your knowledge of generative AI. Well done!

—Chris Fregly, Principal Solution Architect, Generative AI at AWS

In the heart of the GenAI revolution, this indispensable guide masterfully balances theory and practice, navigating the vast landscape of large language models to equip readers with the knowledge needed for immediate and transformative impact in the field of AI.

—Tarun Narayanan Venkatachalam, AI Researcher, University of Washington

Timely reading to get hands-on experience with language models.

—Emir Muñoz, Genesys

Hands-On Large Language Models brings clarity and practical examples to cut through the hype of AI. It provides a wealth of great diagrams and visual aids to supplement the clear explanations. The worked examples and code make concrete what other books leave abstract. The book starts with simple

introductory beginnings, and steadily builds in scope. By the final chapters, you will be fine-tuning and building your own large language models with confidence.

—Leland McInnes, Researcher at the Tutte Institute for Mathematics and Computing

Finally, a book that not only avoids superficial coverage of large language models but also thoroughly explores the background in a way that is both accessible and engaging. The authors have masterfully created a definitive guide that will remain essential reading despite the fast-paced advancements in the field.

—Prof. DDr. Roman Egger, CEO of Smartvisions.at and Modul University Vienna

Hands-On Large Language Models

Language Understanding and Generation

Jay Alammar and Maarten Grootendorst

Hands-On Large Language Models

by Jay Alammar and Maarten Grootendorst

Copyright © 2024 Jay Alammar and Maarten Pieter Grootendorst. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Nicole Butterfield

Development Editor: Michele Cronin

Production Editor: Ashley Stussy

Copyeditor: Charles Roumeliotis

Proofreader: Kim Cofer

Indexer: BIM Creatives, LLC

Interior Designer: David Futato

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

September 2024: First Edition

Revision History for the First Edition

- 2024-09-10: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098150969> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Hands-On Large Language Models*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-15096-9

[LSI]

Preface

Large language models (LLMs) have had a profound and far-reaching impact on the world. By enabling machines to better understand and generate human-like language, LLMs have opened new possibilities in the field of AI and impacted entire industries.

This book provides a comprehensive and highly visual introduction to the world of LLMs, covering both the conceptual foundations and practical applications. From word representations that preceded deep learning to the cutting-edge (at the time of this writing) Transformer architecture, we will explore the history and evolution of LLMs. We delve into the inner workings of LLMs, exploring their architectures, training methods, and fine-tuning techniques. We also examine various applications of LLMs in text classification, clustering, topic modeling, chatbots, search engines, and more.

With its unique blend of intuition-building, applications, and illustrative style, we hope that this book provides the ideal foundation for those looking to explore the exciting world of LLMs. Whether you are a beginner or an expert, we invite you to join us on this journey to start building with LLMs.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

An Intuition-First Philosophy

The main goal of this book is to provide an *intuition* into the field of LLMs. The pace of development in the Language AI field is incredibly fast and frustration can build trying to keep up with the latest technologies. Instead, we focus on the fundamentals of LLMs and intend to provide a fun and easy learning process.

To achieve this *intuition-first philosophy* we liberally make use of visual language. Illustrations will help give a visual identity to major concepts and processes involved in the learning process of LLMs.¹ With our illustrative method of storytelling, we want to take you on a journey to this exciting and potentially world-changing field.

Throughout the book, we make a clear distinction between representation and generative language models. Representation models are LLMs that do not generate text but are commonly used for task-specific use cases, like classification, whereas generation models are LLMs that generate text, like GPT models. Although generative models are typically the first thing that comes to mind when thinking about LLMs, there is still much use for representation models. We are also loosely using the word “large” in *large language models* and often elect to simply call them language models as size descriptions are often rather arbitrary and not always indicative of capability.

Prerequisites

This book assumes that you have some experience programming in Python and are familiar with the fundamentals of machine learning. The focus will be on building a strong intuition rather than deriving mathematical equations. As such, illustrations combined with hands-on examples will drive the examples and learning through this book. This book assumes no prior knowledge of popular deep learning frameworks such as PyTorch or TensorFlow nor any prior knowledge of generative modeling.

If you are not familiar with Python, a great place to start is [Learn Python](#), where you will find many tutorials on the basics of the language. To further ease the learning process, we made all the code available on [Google Colab](#), a platform where you can run all of the code without the need to install anything locally.

Book Structure

The book is broadly divided into three parts. They are illustrated in [Figure P-1](#) to give you a full view of the book. Note that each chapter can be read independently, so feel free to skim chapters you are already familiar with.

Part I: Understanding Language Models

In Part I of the book, we explore the inner workings of language models both small and large. We start with an overview of the field and common techniques (see [Chapter 1](#)) before moving over to two central components of these models, tokenization and embeddings (see [Chapter 2](#)). We finish this part of the book with an updated and expanded version of Jay's well-known [Illustrated Transformer](#), which dives into the architecture of these models (see [Chapter 3](#)). Many terms and definitions will be introduced that are used throughout the book.

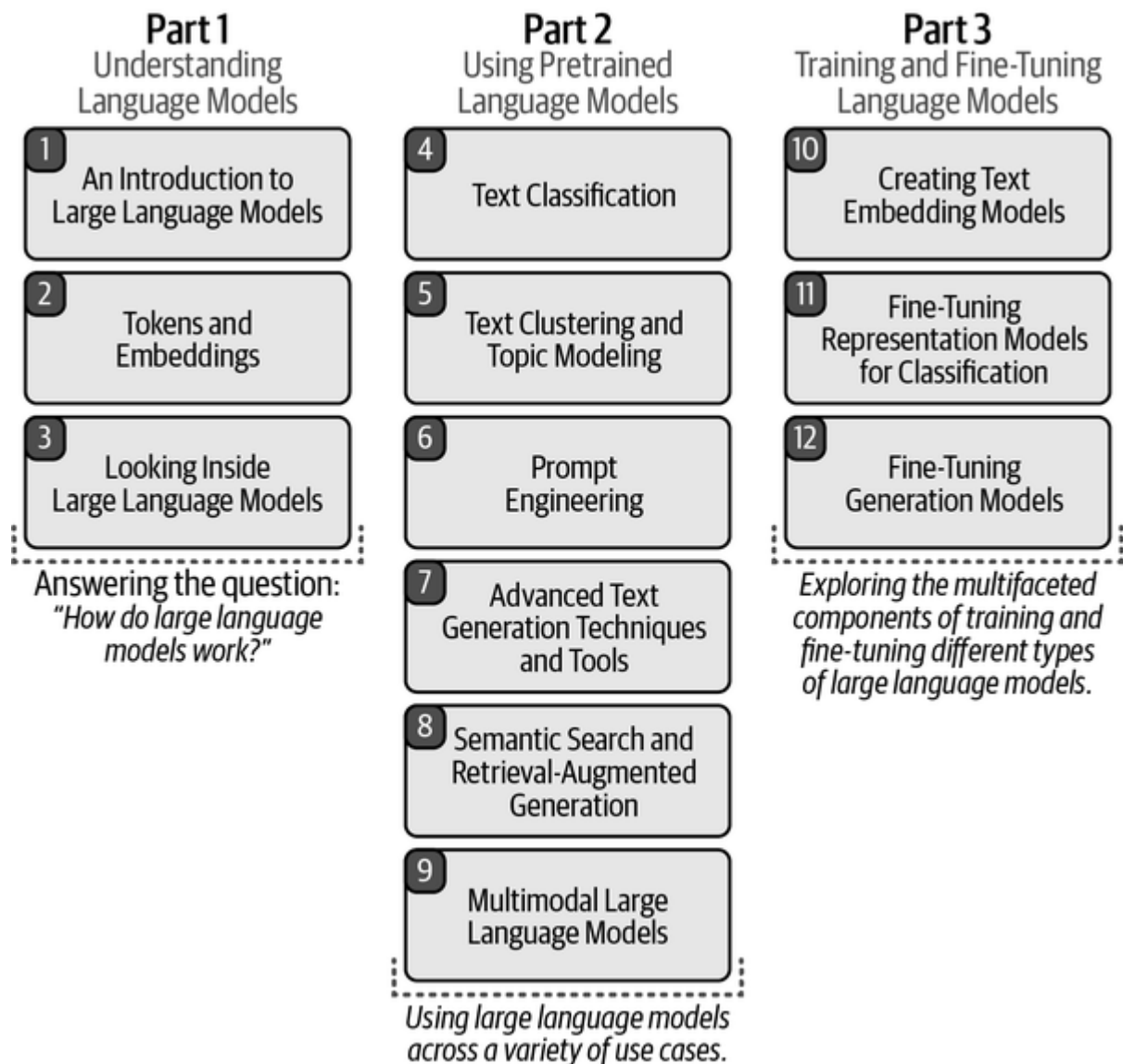


Figure P-1. All parts and chapters of the book.

Part II: Using Pretrained Language Models

In Part II of the book, we explore how LLMs can be used through common use cases. We use pretrained models and demonstrate their capabilities without the need to fine-tune them.

You learn how to use language models for supervised classification (see [Chapter 4](#)), text clustering and topic modeling (see [Chapter 5](#)), leveraging embedding models for semantic search (see [Chapter 6](#)), generating text (see [Chapters 7 and 8](#)), and extending the capabilities of text generation to the visual domain (see [Chapter 9](#)).

Learning these individual language model capabilities will equip you with the skill set to problem-solve with LLMs and build more and more advanced systems and pipelines.

Part III: Training and Fine-Tuning Language Models

In Part III of the book, we explore advanced concepts through training and fine-tuning all kinds of language models. We will explore how to create and fine-tune an embedding model (see [Chapter 10](#)), review how to fine-tune BERT for classification (see [Chapter 11](#)), and end the book with several methods for fine-tuning generation models (see [Chapter 12](#)).

Hardware and Software Requirements

Running generative models is generally a compute-intensive task that requires a computer with a strong GPU. Since those are not available to every reader, all examples in this book are made to run using an online platform, namely [Google Colaboratory](#), often shortened to “Google Colab.” At the time of writing, this platform allows you to use an NVIDIA GPU (T4) for free to run your code. This GPU has 16 GB of VRAM (which is the memory of your GPU), which is the minimum amount of VRAM we expect for the examples throughout the book.

NOTE

Not all chapters require a minimum of 16 GB VRAM as some examples, like training and fine-tuning, are more compute-intensive than others, such as prompt engineering. In the repository, you will find the minimum GPU requirements for each chapter.

All code, requirements, and additional tutorials are available in [this book’s repository](#). If you want to run the examples locally, we recommend access to an NVIDIA GPU with a minimum of 16 GB of VRAM. For a local installation, for example with conda, you can follow this setup to create your environment:

```
conda create -n thellmbook python=3.10
conda activate thellmbook
```

You can install all the necessary dependencies by forking or cloning the repository and then running the following in your newly created Python 3.10 environment:

```
pip install -r requirements.txt
```

API Keys

We use both open source and proprietary models throughout the examples to demonstrate the advantages and disadvantages of both. For the proprietary models, using OpenAI and Cohere's offering, you will need to create a free account:

OpenAI

Click "sign up" on the site to create a free account. This account allows you to create an API key, which can be used to access GPT-3.5. Then, go to "API keys" to create a secret key.

Cohere

Register a free account on the website. Then, go to "API keys" to create a secret key.

Note that with both accounts, rate limits apply and that these free API keys only allow for a limited number of calls per minute. Throughout all examples, we have taken that into account and provided local alternatives if necessary.

For the open source models, you do not need to create an account with the exception of the Llama 2 model in [Chapter 2](#). To use that model, you will need a Hugging Face account:

Hugging Face

Click "sign up" on the Hugging Face website to create a free account. Then, in "Settings" go to "Access Tokens" to create a token that you can use to download certain LLMs.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/HandsOnLLM/Hands-On-Large-Language-Models>.

If you have a technical question or a problem using the code examples, please send email to support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Hands-On Large Language Models* by Jay Alammar and Maarten Grootendorst (O'Reilly). Copyright 2024 Jay Alammar and Maarten Pieter Grootendorst, 978-1-098-15096-9.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://www.oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at https://oreil.ly/hands_on_LLMs_1e.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

Writing this book has been an incredible experience, collaboration, and journey for us.

The field of (large) language models is one of the most dynamic areas in technology today, and within the span of writing this book, we have witnessed extraordinary advancements. Yet, despite the rapid pace of change, the fundamental principles remain strikingly consistent which made the writing process particularly intriguing. We are grateful to have had the opportunity to explore this field in-depth at such a pivotal moment.

Working with our O'Reilly team was incredible! Special thanks to Michele Cronin for her amazing feedback, support, and enthusiasm for this book from day one. We could not have asked for a better

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

editor—you are amazing! Thank you, Nicole Butterfield, for kicking off this book and helping us maintain a structured approach throughout the writing. Thank you to Karen Montgomery for creating our wonderful cover, we love the kangaroo! Big thanks to Kate Dullea for being so patient with us having to go through hundreds of illustrations many times over. The timely early releases by Clare Laylock helped us see our work grow which was a big motivator, thank you. Thanks to Ashley Stussy and Charles Roumeliotis for the development in the final stages of the book and everyone else at O'Reilly who contributed.

Thanks to our amazing crew of technical reviewers. Invaluable feedback was given by Harm Buisman, Emir Muñoz, Luba Elliott, Guarav Chawla, Rafael V. Pierre, Luba Elliott, Tarun Narayanan, Nikhil Buduma, and Patrick Harrison.

Jay

I'd love to extend my deepest gratitude to my family for their unwavering support and inspiration. I would like to specifically acknowledge my parents, Abdullah and Mishael, and my aunts, Hussah and Aljoharah.

I'm grateful to the friends, colleagues, and collaborators who helped me understand and explain the tricky concepts covered in this book as well as to the Cohere folks who cultivate a supporting learning and sharing environment. Thank you to Adrien Morisot, Aidan Gomez, Andy Toulis, Anfal Alatawi, Arash Ahmadian, Bharat Venkitesh, Edward Grefenstette, Ivan Zhang, Joao Araújo, Luis Serrano, Matthias Gallé, Meor Amer, Nick Frosst, Patrick Lewis, Phil Blunsom, Sara Hooker, and Suhas Pai.

I couldn't conceive of this project getting accomplished to the level it has without the extraordinary talent and tireless effort of Maarten, my coauthor. Your ability to repeatedly nail the technical details (from the pinned version of the `nth` import dependency to the latest in LLM quantization) while weaving some of the world's best visual narratives is absolutely breathtaking.

Lastly, a tip of the hat to the incredible coffee shop scene of Riyadh, Saudi Arabia for supplying me with caffeine and a good place to focus from dawn until midnight. It's where I read most of these papers and worked out my understanding (looking at you, Elixir Bunn).

Maarten

I want to begin by expressing my heartfelt appreciation to my coauthor, Jay. Your insights have made this not only possible but incredibly fulfilling. This journey has been nothing short of amazing and collaborating with you has been an absolute joy.

I want to sincerely thank my wonderful colleagues at IKNL for their continued support throughout this journey. A special mention goes to Harm—our Monday morning coffee breaks discussing this book were a constant source of encouragement.

Thank you to my family and friends for their unwavering support, and to my parents in particular. Pap, despite the challenges you faced, you always found a way to be there for me when I needed it most, thank you. Mam, the conversations we had as aspiring writers were wonderful and motivated me more than you could ever imagine. Thank you both for your endless support and encouragement.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Finally, I am at a loss for words to adequately express my gratitude to my wonderful wife, Ilse. Lieverd, your boundless enthusiasm and patience have been legendary, especially when I droned on about the latest LLM developments for hours on end. You are my greatest support. My apologies to my amazing daughter, Sarah. At just two years old, you already have listened to more about large language models than anyone should have to endure in a lifetime! I promise we'll make up for it with endless playtime and adventures together.

¹ J. Alamar. "Machine learning research communication via illustrated and interactive web articles." *Beyond Static Papers: Rethinking How We Share Scientific Understanding in ML*. ICLR 2021 Workshop (2021).

Part I. Understanding Language Models

Chapter 1. An Introduction to Large Language Models

Humanity is at an inflection point. From 2012 onwards, developments in building AI systems (using deep neural networks) accelerated so that by the end of the decade, they yielded the first software system able to write articles indiscernible from those written by humans. This system was an AI model called Generative Pre-trained Transformer 2, or GPT-2. 2022 marked the release of ChatGPT, which demonstrated how profoundly this technology was poised to revolutionize how we interact with technology and information. Reaching one million active users in five days and then one hundred million active users in two months, the new breed of AI models started out as human-like chatbots but quickly evolved into a monumental shift in our approach to common tasks, like translation, text generation, summarization, and more. It became an invaluable tool for programmers, educators, and researchers.

The success of ChatGPT was unprecedented and popularized more research into the technology behind it, namely large language models (LLMs). Both proprietary and public models were being released at a steady pace, closing in on, and eventually catching up to the performance of ChatGPT. It is not an exaggeration to state that almost all attention was on LLMs.

As a result, 2023 will always be known, at least to us, as the year that drastically changed our field, Language Artificial Intelligence (Language AI), a field characterized by the development of systems capable of understanding and generating human language.

However, LLMs have been around for a while now and smaller models are still relevant to this day. LLMs are much more than just a single model and there are many other techniques and models in the field of language AI that are worth exploring.

In this book, we aim to give readers a solid understanding of the fundamentals of both LLMs and the field of Language AI in general. This chapter serves as the scaffolding for the rest of the book and will introduce concepts and terms that we will use throughout the chapters.

But mostly, we intend to answer the following questions in this chapter:

- What is Language AI?
- What are large language models?
- What are the common use cases and applications of large language models?
- How can we use large language models ourselves?

What Is Language AI?

The term *artificial intelligence* (AI) is often used to describe computer systems dedicated to performing tasks close to human intelligence, such as speech recognition, language translation, and visual perception. It is the intelligence of software as opposed to the intelligence of humans.

Here is a more formal definition by one of the founders of the artificial intelligence discipline:

[Artificial intelligence is] the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable.

—John McCarthy, 2007¹

Due to the ever-evolving nature of AI, the term has been used to describe a wide variety of systems, some of which might not truly embody intelligent behavior. For instance, characters in computer games (NPCs [nonplayable characters]) have often been referred to as AI even though many are nothing more than *if-else* statements.

Language AI refers to a subfield of AI that focuses on developing technologies capable of understanding, processing, and generating human language. The term *Language AI* can often be used interchangeably with *natural language processing* (NLP) with the continued success of machine learning methods in tackling language processing problems.

We use the term *Language AI* to encompass technologies that technically might not be LLMs but still have a significant impact on the field, like how retrieval systems can give LLMs superpowers (see [Chapter 8](#)).

Throughout this book, we want to focus on the models that have had a major role in shaping the field of Language AI. This means exploring more than just LLMs in isolation. That, however, brings us to the question: what are large language models? To begin answering this question in this chapter, let's first explore the history of Language AI.

A Recent History of Language AI

The history of Language AI encompasses many developments and models aiming to represent and generate language, as illustrated in Figure 1-1.

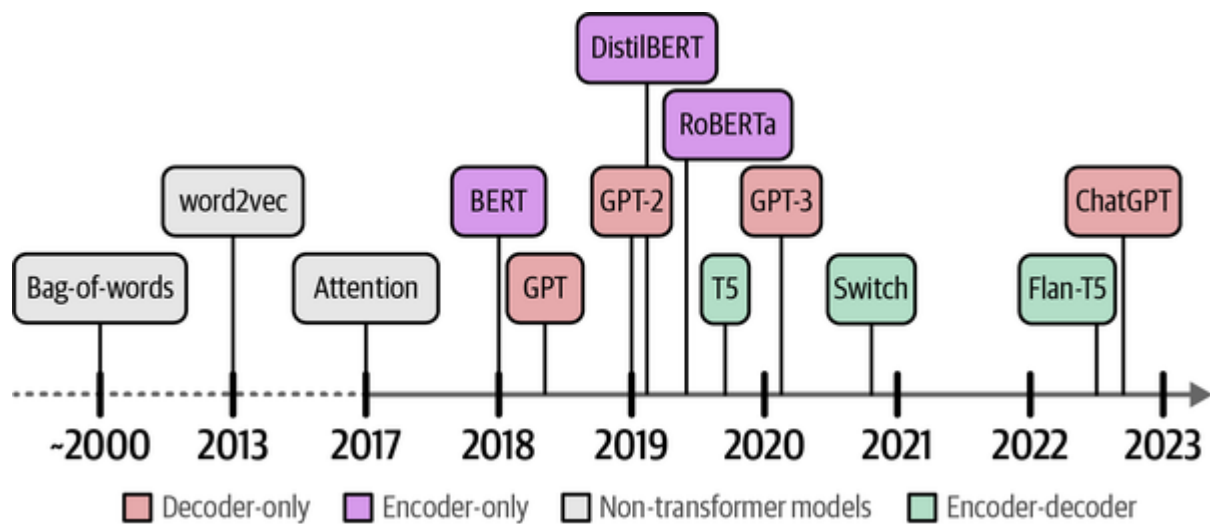


Figure 1-1. A peek into the history of Language AI.

Language, however, is a tricky concept for computers. Text is unstructured in nature and loses its meaning when represented by zeros and ones (individual characters). As a result, throughout the history of Language AI, there has been a large focus on representing language in a structured manner so that it can more easily be used by computers. Examples of these Language AI tasks are provided in Figure 1-2.

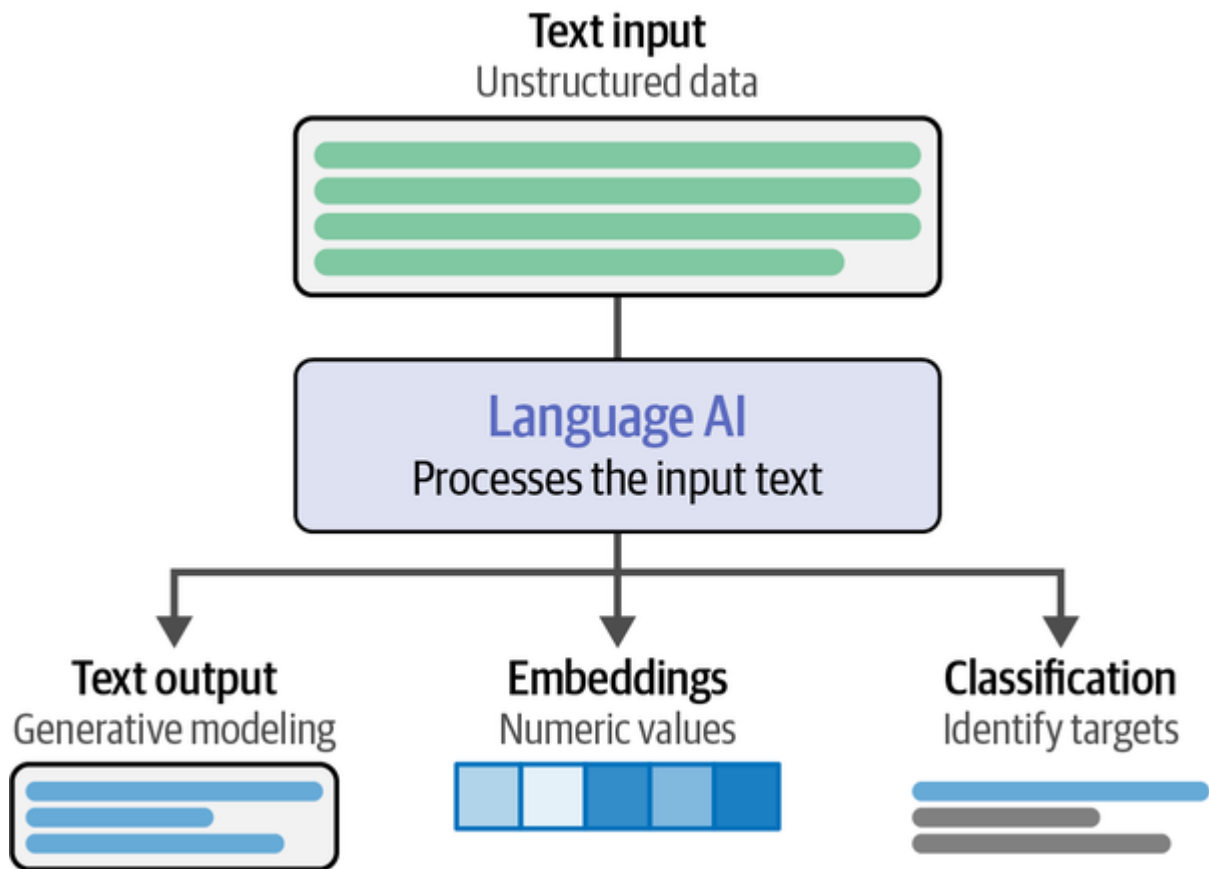


Figure 1-2. Language AI is capable of many tasks by processing textual input.

Representing Language as a Bag-of-Words

Our history of Language AI starts with a technique called bag-of-words, a method for representing unstructured text.² It was first mentioned around the 1950s but became popular around the 2000s.

Bag-of-words works as follows: let's assume that we have two sentences for which we want to create numerical representations. The first step of the bag-of-words model is *tokenization*, the process of splitting up the sentences into individual words or subwords (*tokens*), as illustrated in Figure 1-3.

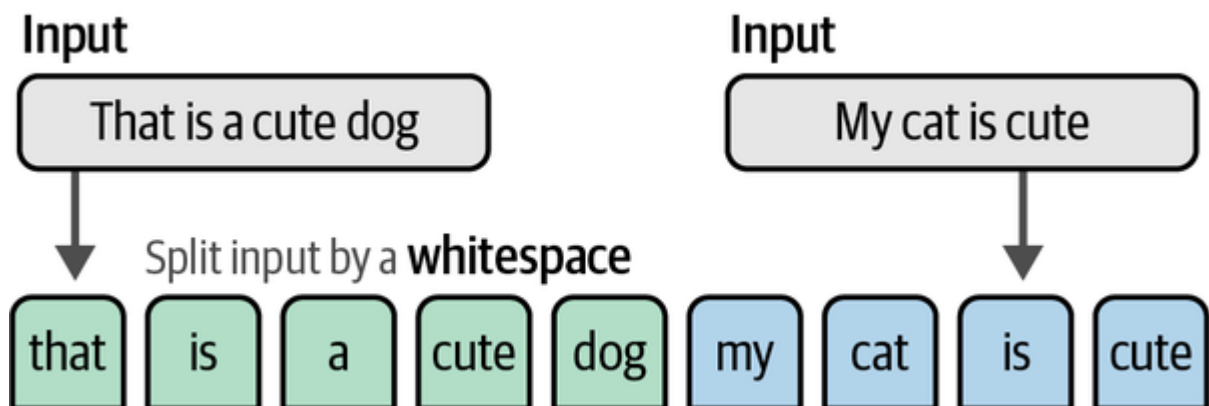


Figure 1-3. Each sentence is split into words (tokens) by splitting on a whitespace.

The most common method for tokenization is by splitting on a whitespace to create individual words. However, this has its disadvantages as some languages, like Mandarin, do not have whitespaces around individual words. In the next chapter, we will go in depth about tokenization and how that technique influences language models. As illustrated in Figure 1-4, after tokenization, we combine all unique words from each sentence to create a vocabulary that we can use to represent the sentences.

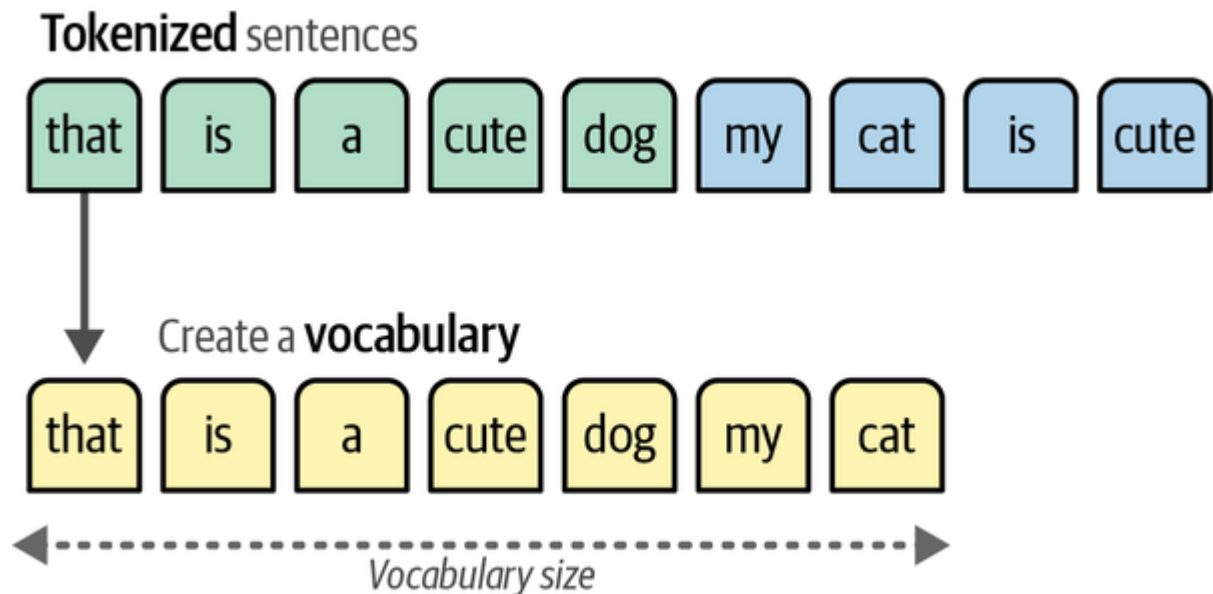


Figure 1-4. A vocabulary is created by retaining all unique words across both sentences.

Using our vocabulary, we simply count how often a word in each sentence appears, quite literally creating a bag of words. As a result, a bag-of-words model aims to create representations of text in the form of numbers, also called vectors or vector representations, observed in Figure 1-5. Throughout the book, we refer to these kinds of models as *representation models*.

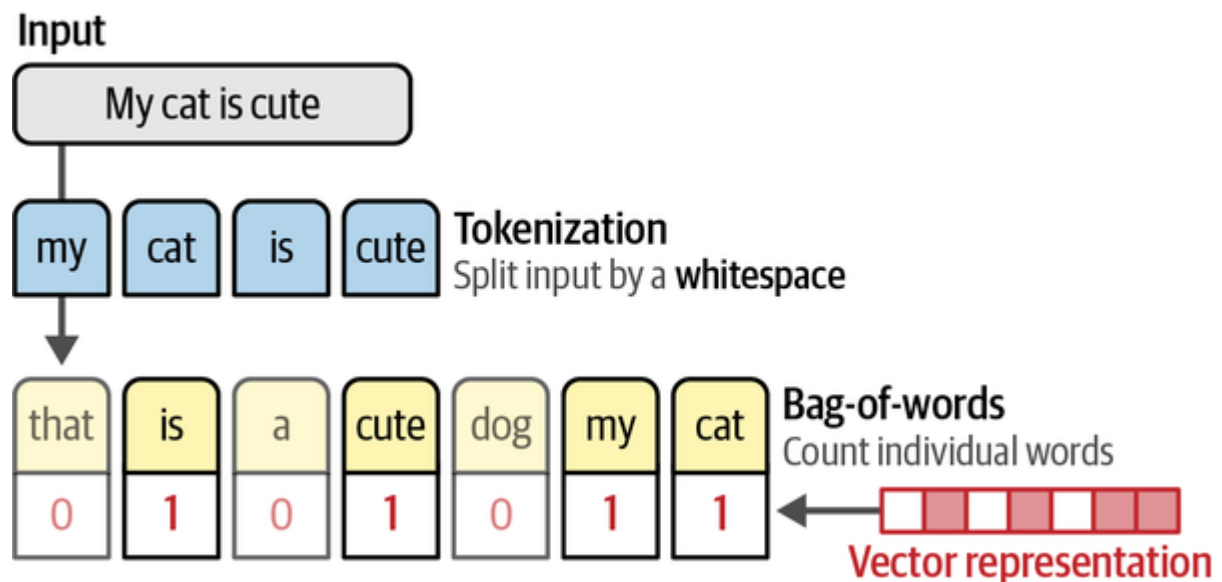


Figure 1-5. A bag-of-words is created by counting individual words. These values are referred to as vector representations.

Although bag-of-words is a classic method, it is by no means completely obsolete. In [Chapter 5](#), we will explore how it can still be used to complement more recent language models.

Better Representations with Dense Vector Embeddings

Bag-of-words, although an elegant approach, has a flaw. It considers language to be nothing more than an almost literal bag of words and ignores the semantic nature, or meaning, of text.

Released in 2013, word2vec was one of the first successful attempts at capturing the meaning of text in *embeddings*.³ Embeddings are vector representations of data that attempt to capture its meaning. To do so, word2vec learns semantic representations of words by training on vast amounts of textual data, like the entirety of Wikipedia.

To generate these semantic representations, word2vec leverages *neural networks*. These networks consist of interconnected layers of nodes that process information. As illustrated in [Figure 1-6](#), neural networks can have many layers where each connection has a certain weight depending on the input. These weights are often referred to as the *parameters* of the model.

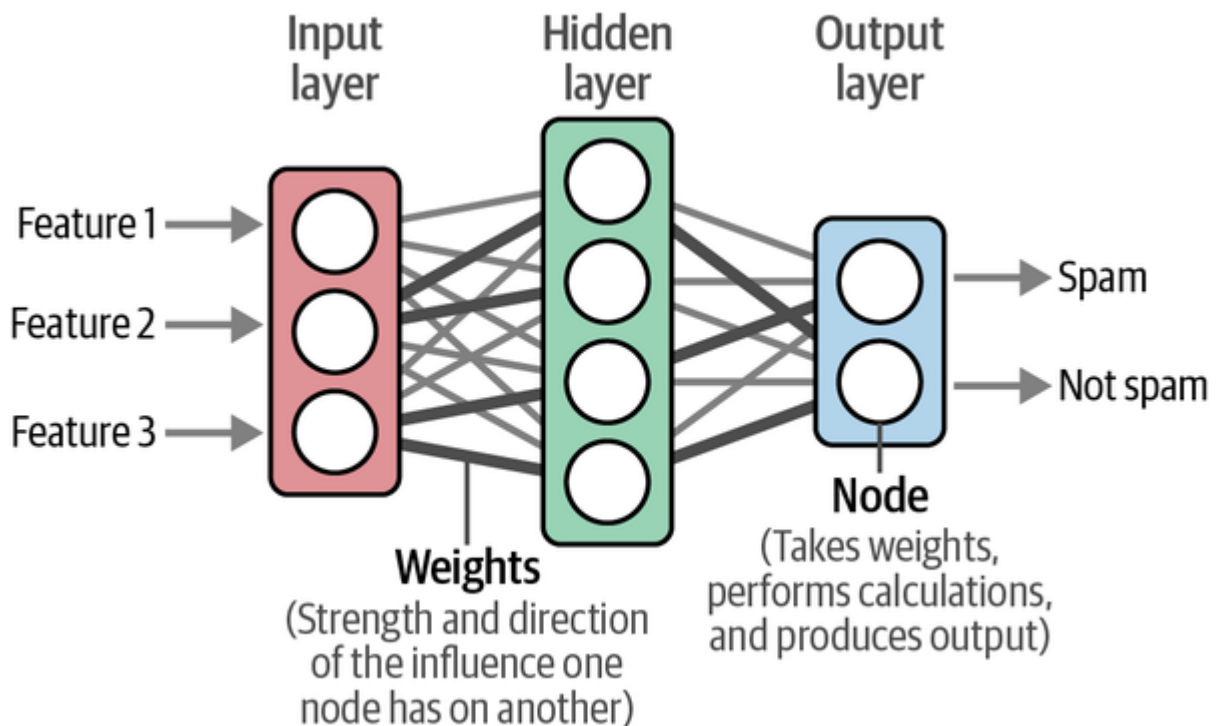


Figure 1-6. A neural network consists of interconnected layers of nodes where each connection is a linear equation.

Using these neural networks, word2vec generates word embeddings by looking at which other words they tend to appear next to in a given sentence. We start by assigning every word in our vocabulary with a vector embedding, say of 50 values for each word initialized with random values. Then in every training step, as illustrated in [Figure 1-7](#), we take pairs of words from the training data and a model attempts to predict whether or not they are likely to be neighbors in a sentence.

During this training process, word2vec learns the relationship between words and distills that information into the embedding. If the two words tend to have the same neighbors, their embeddings will be closer to one another and vice versa. In [Chapter 2](#), we will look closer at word2vec’s training procedure.

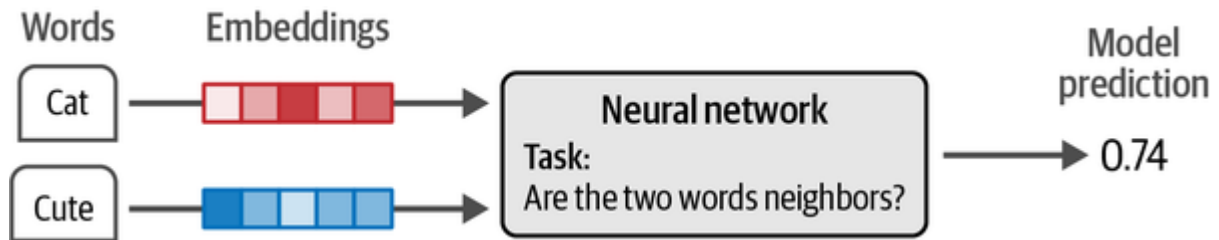


Figure 1-7. A neural network is trained to predict if two words are neighbors. During this process, the embeddings are updated to be in line with the ground truth.

The resulting embeddings capture the meaning of words but what exactly does that mean? To illustrate this phenomenon, let’s somewhat oversimplify and imagine we have embeddings of several words, namely “apple” and “baby.” Embeddings attempt to capture meaning by representing the properties of words. For instance, the word “baby” might score high on the properties “newborn” and “human” while the word “apple” scores low on these properties.

As illustrated in [Figure 1-8](#), embeddings can have many properties to represent the meaning of a word. Since the size of embeddings is fixed, their properties are chosen to create a mental representation of the word.

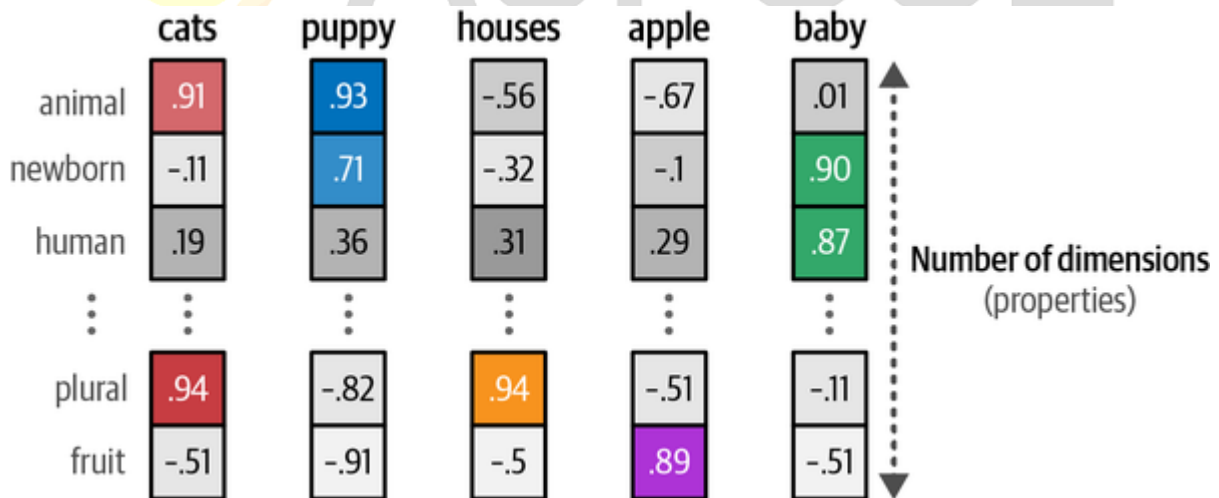


Figure 1-8. The values of embeddings represent properties that are used to represent words. We may oversimplify by imagining that dimensions represent concepts (which they don’t), but it helps express the idea.

This document was truncated here because it was created in the Evaluation Mode.